

Syscall throttling LKM

Greta Ancora 0369455

Indice

1	Specifiche di progetto	3
1.1	Interfaccia e Gestione delle Regole	3
1.2	Logica di Monitoraggio e Throttling	3
1.3	Telemetria e Statistiche	3
1.4	Spazio Utente	4
2	Requisiti tecnici e dipendenze	4
3	Monitoraggio	4
3.1	Patching binario e Shadow system call table	4
3.2	Filtraggio	6
3.2.1	Strutture Dati: Hash Table e liste <i>hlist</i>	6
3.2.2	Slab Allocator per i nodi delle regole	6
3.2.3	Sincronizzazione Lockless tramite RCU	6
4	Throttling	7
4.1	Meccanismo Lockless Token Bucket	7
4.2	Temporizzazione Event-Driven e Workqueue	7
4.3	Waitqueue	8
5	Telemetria e Statistiche	9
5.1	Variabili Per-CPU	9
5.2	Allineamento in Memoria e Prevenzione del False Sharing	9
5.3	Aggregazione Differita: Pattern Map-Reduce	10
6	Interfaccia Utente-Kernel	10
6.1	Implementazione del Driver	11
6.2	Gestione dei Permessi e Privilege Boundary	11
7	Smontaggio del modulo	11
8	Risultati dei test	12
8.1	Analisi Prestazionale	12
8.2	Determinismo del Rate-Limiting e Tolleranza alla Concorrenza	13
8.3	Sicurezza del Control Plane	13
8.4	Stress Test di Hot-Unplugging	13

1 Specifiche di progetto

Il presente documento illustra la progettazione e l'implementazione di un Modulo Kernel Linux (LKM) finalizzato al throttling (limitazione del tasso di esecuzione) delle chiamate di sistema. L'obiettivo principale è fornire un meccanismo di controllo per limitare la frequenza di invocazione di specifiche system call, considerate critiche per ragioni di scalabilità, prestazioni o sicurezza.

1.1 Interfaccia e Gestione delle Regole

Il modulo espone un device driver che funge da interfaccia tra lo spazio utente e lo spazio kernel, supportando operazioni di registrazione e deregistrazione dinamica tramite la chiamata di sistema `ioctl()`. Nello specifico, il sistema permette di monitorare:

- nomi dei programmi
- UID degli utenti
- numeri delle system call (conformi alle specifiche dell'architettura x86_64)

Per ragioni di sicurezza, le operazioni di scrittura e aggiornamento di queste regole sono strettamente riservate a thread in esecuzione con privilegi di amministratore (effective UID uguale a 0, ovvero root). Al contrario, la lettura e la verifica dello stato attuale delle regole (programmi, utenti e syscall registrate) è accessibile in qualsiasi momento a qualunque utente del sistema. Il modulo offre inoltre un interruttore globale per attivare o disattivare il monitoraggio: quando disabilitato, le system call vengono eseguite nativamente senza alcuna limitazione di frequenza.

1.2 Logica di Monitoraggio e Throttling

Il cuore del LKM è un monitor che intercetta le chiamate di sistema. L'intercettazione avviene se e solo se una syscall registrata viene invocata da un programma registrato oppure da un utente registrato.

Il monitor è configurabile per applicare un limite massimo che rappresenta il volume massimo di invocazioni consentite all'interno di una finestra temporale (wall-clock-time) di 1 secondo. Se il tasso di invocazioni supera la soglia, i thread chiamanti vengono temporaneamente bloccati e messi in attesa, prevenendo l'esecuzione effettiva della chiamata di sistema fino al ripristino della finestra temporale. Questa logica è progettata per essere agnostica rispetto alla natura della chiamata di sistema stessa, supportando sia chiamate bloccanti che non bloccanti.

1.3 Telemetria e Statistiche

Inoltre, il device driver raccoglie e fornisce dati statistici e metriche di profilazione in tempo reale basate sul limite massimo impostato. In particolare, il sistema espone:

- il ritardo di picco (peak delay) imposto all'esecuzione effettiva di una chiamata di sistema, associato al nome del programma e all'ID utente che lo ha generato
- il volume dei thread bloccati, espresso sia come media nel tempo sia come picco massimo di thread simultaneamente in attesa

1.4 Spazio Utente

A corredo del Linux Kernel Module, il progetto prevede la consegna di applicativi a livello utente. Questi includono sia gli strumenti di interfaccia a riga di comando per la configurazione dinamica del monitor, sia una suite di test automatizzata, essenziale per validare la correttezza, la robustezza e le prestazioni del software implementato sotto diversi carichi di lavoro.

2 Requisiti tecnici e dipendenze

L'integrità e le prestazioni del modulo sono garantite per i seguenti ambienti di riferimento:

- Architettura Target: x86_64
- Versione Kernel: Linux 6.x o superiore
- Toolchain: GCC 12.1.0+ e Kernel Headers aggiornati alla versione del sistema in esecuzione
- Configurazione Kernel Richiesta (.config):
 - CONFIG_MODULES=y: indispensabile per la compilazione e il caricamento dinamico del driver
 - CONFIG_KPROBES=y e CONFIG_KALLSYMS=y: necessari per l'individuazione dinamica dell'indirizzo del dispatcher `x64_sys_call()` bypassando la restrizione introdotta nel kernel 5.7 sull'esportazione di `kallsyms_lookup_name()`
 - CONFIG_SMP=y: necessario per la corretta gestione delle variabili Per-CPU
 - CONFIG_RETPOLINE=y: necessario per la mitigazione di Spectre v2

3 Monitoraggio

3.1 Patching binario e Shadow system call table

Il sistema di monitoraggio delle system call è stato implementato tramite il patching binario del dispatcher di alto livello `x64_sys_call()` (per le chiamate definite in `unistd64`) combinato all'utilizzo di una shadow system call table.

A partire dal kernel 5.15, la tabella delle system call originale non viene più interrogata dinamicamente dal dispatcher principale: il kernel genera un codice ottimizzato sotto forma di costrutto `switch`, passando direttamente all'esecuzione dei security incapsulators. Il patching binario di `x64_sys_call()` risulta pertanto strettamente necessario per deviare il flusso di esecuzione e reindirizzarlo verso la nostra shadow table globale. Tale dirottamento viene effettuato iniettando un trampolino assembly all'inizio del dispatcher.

La shadow system call table viene inizializzata in fase di caricamento del modulo, clonando la system call table globale `unistd64` (mantenuta in memoria nel kernel per ragioni di retrocompatibilità). L'indirizzo di base di quest'ultima viene individuato dinamicamente sfruttando l'algoritmo di discovery fornito dal Prof. Francesco Quaglia, basato sull'euristica di posizionamento delle entry relative alla chiamata `sys_ni_syscall()`.

Durante la registrazione di una system call critica tramite `ioctl()`, la relativa entry all'interno della shadow table viene modificata per puntare alla funzione `targeted_monitor_entry()`. Quest'ultima verifica le regole di filtraggio relative a UID e programma e se le condizioni sono soddisfatte instrada l'esecuzione verso il fast-path del meccanismo di throttling. In fase di deregistrazione, la entry della shadow table viene ripristinata ricopiando il function pointer dalla tabella originale del kernel.

Questo meccanismo garantisce un monitoraggio a grana fine (per singola system call), mantenendo l'overhead a zero per le chiamate non intercettate. Inoltre, l'utilizzo di una shadow table autonoma evita la necessità di alterare a run-time la tabella originale del kernel. Modificare quest'ultima richiederebbe continue scritture sui registri di controllo `cr0` e `cr4` per disabilitare momentaneamente i meccanismi di protezione della memoria read-only (WP = Write Protect, bit 16 di `cr0`) e i controlli CET (Control-flow Enforcement Technology, bit 23 di `cr4`), operazioni che causerebbero in caso di massive operazioni di registrazione e deregistrazione da parte dello spazio utente:

- potenziale instabilità: invalidazione costante in cache L1 delle entries originali della syscall table, che porta a un calo di prestazioni delle chiamate non registrate
- stallo del sistema: possibile fenomeno di *Interrupt Jitter*, dovuto alla necessità rendere il thread che si occupa di scrivere sulla memoria del kernel unpreemptable e uninterruptible

Durante lo sviluppo della logica di monitoraggio sono stati presi in considerazione aspetti relativi alla sicurezza, in particolare alla mitigazione degli attacchi basati su esecuzione speculativa (famiglia Spectre):

1. **Mitigazione Spectre v1:** Nel trampolino assembly è stata implementata una barriera speculativa equivalente alla macro C `array_index_nospec`, sfruttando lo stato del `Carry Flag` per sanitizzare il valore di `rax` e impedire la lettura di memoria speculativa tramite la shadow table. La

medesima protezione è applicata nel codice C prima di richiamare la funzione originale del kernel e di procedere con l'hook/unhook di una system call.

2. **Mitigazione Spectre v2:** Nel trampolino assembly, la chiamata effettiva alla system call avviene sfruttando un **Retpoline** (tramite il thunk `_x86_indirect_thunk_rax`), rendendo il salto indiretto immune all'avvenimento del Branch Target Buffer (BTB).

3.2 Filtraggio

Il filtraggio delle chiamate di sistema avviene seguendo rigorosamente la logica descritta nelle specifiche di progetto. Poiché la routine di verifica `should_monitor_target()` si trova sul percorso critico (*fast-path*) di ogni singola system call intercettata, si è cercato di minimizzare la latenza introdotta. Per rendere questa operazione il più efficiente possibile e scalabile su architetture multi-core (SMP), si è progettata un'architettura basata su Hash Table dedicate, allocazione personalizzata tramite Slab Allocator e sincronizzazione *lockless* tramite il meccanismo RCU (Read-Copy-Update).

3.2.1 Strutture Dati: Hash Table e liste *hlist*

Per la registrazione dinamica degli UID e dei nomi dei programmi sono state istanziate due Hash Table globali distinte, sfruttando le primitive standard del kernel Linux. La dimensione di ciascuna tabella è stata impostata tramite la macro `DEFINE_HASHTABLE` con un parametro `bits` pari a 8, che si traduce in un'allocazione esatta di $2^8 = 256$ buckets. In caso di collisioni, il kernel gestisce l'accodamento tramite le strutture `hlist_node`. A differenza delle classiche `list_head` (liste doppiamente concatenate standard), le `hlist` prevedono una testa a puntatore singolo (`hlist_head`). Questo accorgimento dimezza l'occupazione di memoria della directory principale dell'Hash Table, massimizzando l'efficienza spaziale e preservando le linee di cache.

3.2.2 Slab Allocator per i nodi delle regole

Per l'istanziatura fisica dei nodi da inserire nelle Hash Table, si è optato per la creazione di una *Cache Slab* dedicata tramite l'API del kernel `kmem_cache_create()`.

L'allocatore Slab del kernel pre-alloca ampie porzioni di memoria fisicamente contigua, dividendole in slot dell'esatta dimensione della struttura dati che rappresenta la singola regola.

3.2.3 Sincronizzazione Lockless tramite RCU

Il cuore dell'efficienza del *fast-path* risiede nel meccanismo RCU (*Read-Copy-Update*). In un sistema multi-core l'utilizzo di uno spinlock o di primitive bloc-

canti in fase di lettura per proteggere le Hash Table causerebbe un significativo fenomeno di contesa.

Il pattern RCU risolve questa criticità architetturale separando le logiche di lettura da quelle di scrittura:

- **Lato Lettori (Fast-Path):** Le decine di migliaia di chiamate di sistema valutano le regole attraversando la Hash Table avvolte dalle macro `rcu_read_lock()` e `rcu_read_unlock()`, che rispettivamente disabilitano e abilitano la preemption locale. I lettori attraversano i nodi tramite `hlist_for_each_entry_rcu()`, che integra speciali barriere di memoria per garantire che il compilatore e l'hardware non riordinino le istruzioni di fetch, assicurando sempre la lettura di puntatori validi, anche se la struttura sta cambiando in quello stesso istante.
- **Lato Scrittori (CLI Config):** Le operazioni di aggiunta o rimozione delle regole acquisiscono un mutex. Supponendo che le operazioni di scrittura avvengano sporadicamente rispetto alle operazioni di lettura, questo meccanismo consente di mitigare i colli di bottiglia dei classici meccanismi di sincronizzazione. L'aggiornamento avviene pubblicando i nuovi puntatori atomicamente. Durante la fase di rimozione, il nodo viene eliminato logicamente dalla lista, ma la sua memoria non viene deallocata immediatamente nella Slab Cache. Il kernel invoca `synchronize_rcu()`, mettendo il thread scrittore in attesa fino a quando tutti i lettori attivi che possedevano la vecchia versione del puntatore non sono usciti dal blocco critico (*Grace Period*). Solo allora la memoria viene reclamata, eliminando qualsiasi rischio di *Use-After-Free*.

4 Throttling

4.1 Meccanismo Lockless Token Bucket

La logica di monitoraggio si occupa di limitare il numero di esecuzioni di chiamate di sistema critiche nell'arco di una finestra temporale di 1 secondo. A tale scopo è stato implementato un meccanismo *lockless token bucket*: ogni thread che necessita di eseguire una system call critica deve prelevare un gettone da un bucket virtuale. Tale operazione avviene in regime di concorrenza sfruttando una variabile `atomic_t` e invocando `atomic_dec_if_positive()`. Sotto l'architettura x86_64, questa direttiva si basa sull'istruzione hardware `LOCK CMPXCHG`, una tipica operazione atomica Read-Modify-Write sul bus della memoria. Questa scelta lockless consente di avere effettiva concorrenza nel tentativo di accesso alla risorsa, evitando i colli di bottiglia prestazionali dei classici meccanismi di lock.

4.2 Temporizzazione Event-Driven e Workqueue

Le epoche da 1 secondo sono scandite attraverso un High-Resolution Timer, il quale viene armato in maniera *lazy*: l'attivazione avviene unicamente nel mo-

mento in cui il primo gettone viene acquisito, ovvero il contatore atomico globale passa da `max_limit` a `max_limit-1`. Questo approccio *event-driven* garantisce che nessun thread rimanga bloccato in coda a tempo indeterminato e, simultaneamente, evita di sprecare cicli di CPU per eseguire routine di temporizzazione quando il sistema non riceve richieste, abbattendo l'impatto complessivo sul sistema. Il top-half dell'hrtimer (eseguito in contesto di Hard IRQ) si limita a programmare l'esecuzione del proprio bottom-half delegandolo a una *workqueue*. Questa separazione risulta necessaria nello scenario in cui dovessimo risvegliare un numero notevole di thread, scorrendo la waitqueue e modificando lo stato dei processi da `TASK_INTERRUPTIBLE` a `TASK_RUNNING`. Se svolgessimo queste operazioni direttamente nel top-half gli interrupt hardware della CPU potrebbero rimanere disabilitati troppo a lungo, ciò comporterebbe gravi criticità, tra cui la perdita di pacchetti di rete, macroscopiche latenze nei dispositivi di input e, nei casi peggiori, il crash del sistema operativo (Watchdog Panic) causato dall'apparente stallo del processore. Nel bottom-half (eseguito in process context), il bucket viene riempito per la nuova epoca e vengono risvegliati al massimo `max_limit` thread dalla waitqueue. Per garantire la massima resilienza, il sistema implementa inoltre una strategia di *Wakeup Propagation* per risolvere il cosiddetto *Lost Wakeup Problem*. In una coda a risveglio esclusivo, si potrebbe verificare uno stallo se i thread risvegliati dalla workqueue venissero terminati da un segnale asincrono (es. `SIGKILL`) prima di poter effettivamente consumare un gettone. In questo scenario, i gettoni rimarrebbero inutilizzati nel bucket e, mancando il prelievo del gettone `max_limit-1`, il timer non verrebbe mai riarmato, lasciando i restanti thread in coda in un'attesa infinita. Per ovviare a ciò, ogni thread che interrompe l'attesa a causa di un segnale ha l'obbligo di propagare il risveglio al processo successivo nella coda (`wake_up_nr()`). Questa catena di risvegli assicura che i gettoni disponibili vengano sempre processati da un thread, garantendo il ripristino deterministico del ciclo di temporizzazione anche in presenza di terminazioni massive.

4.3 Waitqueue

La messa in sleep dei thread nello slow path avviene attraverso una waitqueue globale (`monitor_wq`) in cui vengono accodati i task che non sono riusciti ad acquisire un token. L'accodamento avviene mediante l'API `prepare_to_wait_exclusive()`. L'utilizzo del flag `exclusive` è cruciale per prevenire il fenomeno del *Thundering Herd*: allo scadere del secondo, la funzione `wake_up_nr()` risveglierà in blocco solo il numero esatto di thread consentito dalla soglia `max_limit`, lasciando in attesa gli altri. Inoltre, l'accodamento in modalità esclusiva implementa nativamente una politica FIFO. Ciò garantisce una *fairness* assoluta ed evita fenomeni di *starvation* per i task più vecchi, giustificando la scelta architetturale, anche a costo di tollerare lievi inversioni di priorità rispetto ai valori di scheduling standard dei processi.

5 Telemetria e Statistiche

La raccolta di metriche prestazionali all'interno di un Modulo Kernel pone sfide architetturali non indifferenti. Poiché l'aggiornamento dei contatori avviene sul percorso critico (*fast-path*) e durante le routine di sospensione (*slow-path*) del monitor, l'utilizzo di primitive di sincronizzazione globali (come *spinlock* o *mutex*) per proteggere variabili condivise introdurrebbe un collo di bottiglia sistemico, vanificando l'efficienza del *lockless token bucket*.

Per risolvere questa criticità ed estrarre le statistiche richieste dalle specifiche senza impattare sul throughput delle chiamate di sistema, l'architettura di profilazione è stata progettata attorno a tre concetti chiave: allocazione Per-CPU, allineamento in memoria della Cache L1 e aggregazione differita tramite pattern Map-Reduce.

5.1 Variabili Per-CPU

Invece di allocare un'unica istanza globale della struttura dati contenente la telemetria (`struct monitor_telemetry`), il modulo utilizza la macro `DEFINE_PER_CPU()` per istruire il kernel ad allocare un'istanza indipendente e speculare di tale struttura per ogni core fisicamente presente nel sistema.

Durante l'esecuzione, quando un thread necessita di aggiornare un contatore, il codice invoca funzioni come `this_cpu_ptr()` o `this_cpu_inc()`. Queste direttive si traducono in istruzioni Assembly che operano sfruttando il registro di segmento `GS` (su architettura x86_64), il quale punta all'area di memoria esclusiva della CPU corrente. Poiché ogni processore scrive esclusivamente nella propria area di memoria privata la contesa sul bus di memoria inter-core è nulla; ciò consente di eliminare l'utilizzo dei lock.

5.2 Allineamento in Memoria e Prevenzione del False Sharing

Allocare variabili Per-CPU non è tuttavia sufficiente per garantire l'immunità dai cali prestazionali hardware. Un aspetto da tenere in conto nelle architetture SMP è il fenomeno del *False Sharing*.

Dato che la struttura dati della telemetria è più piccola di 64 byte (dimensione tipica delle linee di cache in x86_64), se le istanze Per-CPU di due core diversi finissero allocate consecutivamente all'interno della medesima linea di cache, si verificherebbe un calo prestazionale: ogni volta che la CPU 0 aggiorna le sue statistiche, il protocollo di coerenza della cache (MESI in x86_64) invaliderebbe forzatamente la copia in Cache L1 della CPU 1 (ritenendola "dirty"), costringendola a ricaricare l'intera linea di cache dalla memoria centrale, causando stalli ciclici del processore (*Cache-Line Bouncing*).

Per scongiurare questo scenario, la struttura dati `monitor_telemetry` è stata esplicitamente allineata in memoria utilizzando l'attributo GCC `___cacheline_aligned`. Questa direttiva impone al compilatore e all'allocatore del kernel di far iniziare ogni istanza Per-CPU esattamente all'inizio di una nuova linea di

cache, inserendo byte di riempimento (*padding*) se necessario. In questo modo, si garantisce l'isolamento fisico assoluto dei dati a livello hardware, permettendo ai core di aggiornare le statistiche in completa indipendenza alla massima velocità.

5.3 Aggregazione Differita: Pattern Map-Reduce

L'utilizzo delle variabili Per-CPU frammenta inevitabilmente la visione globale del sistema. Per restituire un report coerente allo spazio utente il driver implementa un'aggregazione differita basata su un paradigma *Map-Reduce*.

- **Fase di Map:** Avviene in tempo reale durante il normale funzionamento del modulo. I vari processori "mappano" gli eventi sui propri contatori locali senza coordinamento globale.
- **Fase di Reduce:** Viene innescata *on-demand* unicamente quando un processo utente esegue una chiamata di sistema `read`. L'handler del device driver scorre tutte le CPU disponibili sfruttando il macro-iteratore del kernel `for_each_possible_cpu()`. Durante l'iterazione il modulo:
 1. somma linearmente i valori cumulativi (es. accumulando il totale dei thread attualmente bloccati per calcolare la media nel tempo)
 2. applica una funzione di massimizzazione sui valori di picco (es. confrontando il *peak delay* di ogni CPU per individuare il ritardo massimo assoluto registrato nell'intero sistema), estraendo contestualmente l'UID e il Program Name associati a tale picco locale

Questa strategia progettuale accetta un compromesso noto come *Eventual Consistency*: i report potrebbero non riflettere gli ultimi aggiornamenti delle strutture di telemetria. Tuttavia, se non vengono effettuati ulteriori campionamenti, il report convergerà verso uno stato finale contenente tutte le statistiche raccolte localmente dalle CPU. Questo grado di tolleranza risulta accettabile per le operazioni di telemetria, in quanto privilegia in modo assoluto le performance del modulo rispetto alla coerenza transazionale della statistica.

6 Interfaccia Utente-Kernel

Per consentire la configurazione dinamica del monitor e l'estrazione della telemetria senza la ricompilazione o il ricaricamento del modulo, si è progettata un'interfaccia di comunicazione basata su un *Character Device Driver*. Questo componente agisce da ponte tra lo spazio utente (*User-Space*) e lo spazio kernel (*Kernel-Space*), esponendo un nodo dispositivo virtuale (`/dev/syscall_monitor`) attraverso il quale gli applicativi possono inviare comandi e leggere lo stato del sistema.

L'implementazione del driver si articola sull'istanziamento di una struttura `file.operations`, all'interno della quale sono stati registrati gli handler personalizzati per le chiamate di sistema `read`, `open`, `release` e `unlocked_ioctl`.

6.1 Implementazione del Driver

Il meccanismo principale per la mutazione dello stato del modulo è delegato all'interfaccia `ioctl()`. A differenza delle classiche operazioni di read/write basate su stream di byte, `ioctl()` permette il passaggio strutturato di comandi e payload complessi tramite un singolo context switch.

I dati forniti dagli applicativi utente (es. la stringa contenente il nome del programma da monitorare) risiedono in uno spazio di indirizzamento virtuale non fidato. La dereferenziazione diretta di tali puntatori causerebbe violazioni di sicurezza critiche e l'intervento delle protezioni hardware SMAP (Supervisor Mode Access), risultando in un Kernel Panic. Pertanto, il driver fa rigoroso uso delle primitive `copy_from_user()` e `copy_to_user()`. Queste funzioni copiano i dati in un buffer sicuro allocato nello stack o nell'heap del kernel, gestendo internamente e in modo trasparente gli eventuali *Page Fault* causati da puntatori utente non validi o paged-out.

6.2 Gestione dei Permessi e Privilege Boundary

Le specifiche di progetto delineano una chiara demarcazione perimetrale dei privilegi: le operazioni di modifica della configurazione devono essere ad appannaggio esclusivo dell'amministratore di sistema (*root*), mentre la modifica dello stato acceso/spento e la lettura della telemetria possono essere consentite ad utenze non privilegiate.

La validazione dei permessi è stata implementata direttamente all'ingresso delle routine del device driver. Prima di processare qualsiasi comando `ioctl()` che comporti un'alterazione della configurazione del monitor il driver interroga i descrittori di sicurezza del thread chiamante tramite l'API `current_euid().val`. Se l'UID effettivo risulta diverso da 0, la richiesta viene immediatamente rigettata restituendo il codice di errore standard `-EPERM` (Operation not permitted).

7 Smontaggio del modulo

Rimuovere un modulo kernel a caldo (hot-unplugging) è un'operazione intrinsecamente critica. Il rischio principale in questa fase è lo scenario di *Use-After-Free*: se il sistema operativo dealloca la memoria contenente il codice del modulo mentre alcuni thread utente lo stanno ancora attraversando, o mentre sono addormentati in attesa al suo interno, la CPU cercherà di eseguire istruzioni ormai inesistenti. Il risultato è un Page Fault in spazio kernel, che si traduce in un Kernel Panic. Per scongiurare questa eventualità e garantire una tolleranza ai guasti assoluta, la procedura di smontaggio è orchestrata attraverso una rigorosa pipeline a fasi, coordinata da un meccanismo lockless di *Active Hazard Tracking*.

Per determinare con esattezza quanti thread si trovassero all'interno dell'hook in un dato istante, senza però penalizzare le performance del sistema è stata adoperata una variabile Per-CPU (un contatore `in_flight` indipendente

per ogni processore). Quando un thread entra nell'hook, incrementa esclusivamente il contatore della propria CPU e lo decrementa un attimo prima di effettuare la return. Questo approccio fornisce una certezza matematica sulle presenze nel modulo, inserendo un overhead prestazionale praticamente nullo.

Quando l'amministratore invoca il comando di rimozione `rmmod`, la funzione di exit del modulo avvia la sequenza di spegnimento settando la variabile globale `module_is_unloading` a `true`. Da questo momento, qualsiasi thread in ingresso rileva il segnale di chiusura imminente, ignora la logica di monitoraggio e salta direttamente all'esecuzione della syscall originale. Immediatamente dopo, il modulo si isola dallo User-Space disattivando l'interfaccia del driver, impedendo così l'arrivo di nuove configurazioni tramite `ioctl()`.

A questo punto, il codice del modulo viene scollegato fisicamente dal sistema operativo. Il dispatcher originale del kernel viene ripristinato sovrascrivendo a runtime il nostro trampolino assembly. Una volta rimosso il salto incondizionato, abbiamo la garanzia fisica che nessun nuovo processo potrà più fare il suo ingresso nel nostro codice.

Resta tuttavia la criticità di dover evacuare i thread rimasti intrappolati all'interno, in particolare quelli sospesi nello slow-path. Per liberarli, il Gatekeeper assegna il valore massimo possibile al secchio dei token e viene lanciato un segnale di risveglio globale su tutta la waitqueue. I thread addormentati si risvegliano, si accorgono dello stato di smontaggio in corso, decrementano regolarmente il loro contatore `in_flight`. Contestualmente, il timer ad alta risoluzione viene cancellato e si attende in modo sincrono che eventuali Bottom Half della workqueue terminino il loro lavoro.

Confinati i thread verso l'uscita, il modulo entra nella vera e propria fase di quiescenza. Il driver avvia un ciclo di attesa: scorre tutte le CPU del sistema e somma i rispettivi contatori `in_flight`. Finché questa somma globale è maggiore di zero, il modulo deduce che c'è ancora qualcuno in volo e cede l'esecuzione addormentandosi per brevi intervalli da 10 millisecondi.

Solo quando la somma degli `in_flight` restituisce zero, il modulo ha la certezza matematica di essere disabitato. A questo punto si distruggono in sicurezza le strutture dati e le configurazioni. Prima di deallocare definitivamente le Slab Caches dalla memoria, viene invocata un'ultima barriera di salvaguardia, la `rcu_barrier()`. Questa funzione sospende l'esecuzione finché il kernel non ha processato tutte le callback di liberazione della memoria RCU pendenti.

Questa architettura consente di gestire in totale sicurezza lo scaricamento a caldo anche in condizioni di massiccia concorrenza e stress del sistema.

8 Risultati dei test

8.1 Analisi Prestazionale

Al fine di valutare l'impatto reale dell'architettura sul fast-path delle system call, è stata condotta un'analisi statistica su 50 iterazioni del micro-benchmark. Ogni iterazione ha misurato il tempo di esecuzione di 1.000.000 di invocazioni

consecutive di una system call cavia (`SYS_getuid()`), prima in modalità nativa e successivamente sottoposta ad intercettazione tramite Shadow Table.

I risultati empirici hanno dimostrato l'efficienza dell'architettura proposta: il modulo introduce un overhead assoluto medio di appena 9.41 ns nanosecondi per singola system call. La deviazione standard registrata (± 10.53 ns) è fisiologica ed è imputabile alla presenza di rari outlier (ritardi di poche decine di nanosecondi dovuti alla preemption dello scheduler del kernel e a cache-miss durante l'esecuzione del test). In conclusione, il design lockless adottato rende il sistema di monitoraggio statisticamente invisibile al normale flusso di esecuzione del sistema operativo, garantendo al contempo un rate-limiting rigido.

8.2 Determinismo del Rate-Limiting e Tolleranza alla Concorrenza

Per validare la tenuta del meccanismo lockless in scenari di stress, il sistema è stato configurato con un limite artificiale restrittivo di 10 system call al secondo. Al fine di indurre uno scenario di Thundering Herd, la suite di collaudo ha generato 25 thread simultanei, programmati per collidere nell'esatto istante nell'invocazione della medesima chiamata di sistema.

Il test si è concluso in esattamente 2.00 secondi. Questo risultato empirico certifica due aspetti architetturali cruciali: in primo luogo, l'operazione `atomic_dec_if_positive()` gestisce in totale sicurezza la contesa simultanea sul bus della memoria, impedendo qualsiasi race condition nella sottrazione dei gettoni; in secondo luogo, l'impiego della waitqueue esclusiva unita al risveglio mirato delegato alla workqueue (`wake_up_nr()`) assorbe perfettamente le ondate di richieste in eccesso, garantendo che nessun thread eluda il limite imposto.

8.3 Sicurezza del Control Plane

Il modulo espone l'interfaccia `/dev/syscall_monitor` con permessi 0666 per permettere la lettura globale delle statistiche e l'attivazione/disattivazione del monitor. Tuttavia, è imperativo che le operazioni di scrittura siano protette. Il test di sicurezza ha simulato un attacco forzando un processo figlio a degradare i propri privilegi (EUID 1000) e tentare una `ioctl()` di aggiornamento del limite. La chiamata è stata correttamente intercettata a livello kernel e respinta con codice d'errore `-EPERM` (Operation not permitted), dimostrando che il control-plane è blindato e accessibile esclusivamente all'utente root (EUID 0).

8.4 Stress Test di Hot-Unplugging

Infine, la tolleranza ai guasti è stata validata tramite uno script bash dedicato (`test_hot_unload.sh`). Sono stati generati molteplici processi spammer concorrenti, appositamente portati a esaurire i token e a rimanere bloccati nella waitqueue dello slow-path. Durante questo stato di blocco globale, è stato forzato lo scaricamento a caldo del modulo tramite `rmmod`. Grazie all'implementazione della pipeline di spegnimento Fail-Safe, il kernel ha rilasciato tutti

i thread in modo ordinato. Nessun processo utente ha subito un Segmentation Fault e il sistema operativo ha mantenuto una perfetta stabilità senza incorrere in un Kernel Panic.